Ansible desde Cero: Automatiza Linux y Servidores

Cursos PIT - 10ma Edicion



Instructor: Cristian Adrian Diaz Garcia

Fecha: 21 de agosto de 2026

UNIVERSIDAD NACIONAL DE INGENIERÍA

CURSOS GRATUITOS

OTI UNI

TRANSFORMACIÓN digital

SESIÓN 5

ROLES, MONITOREO Y BUENAS PRÁCTICAS

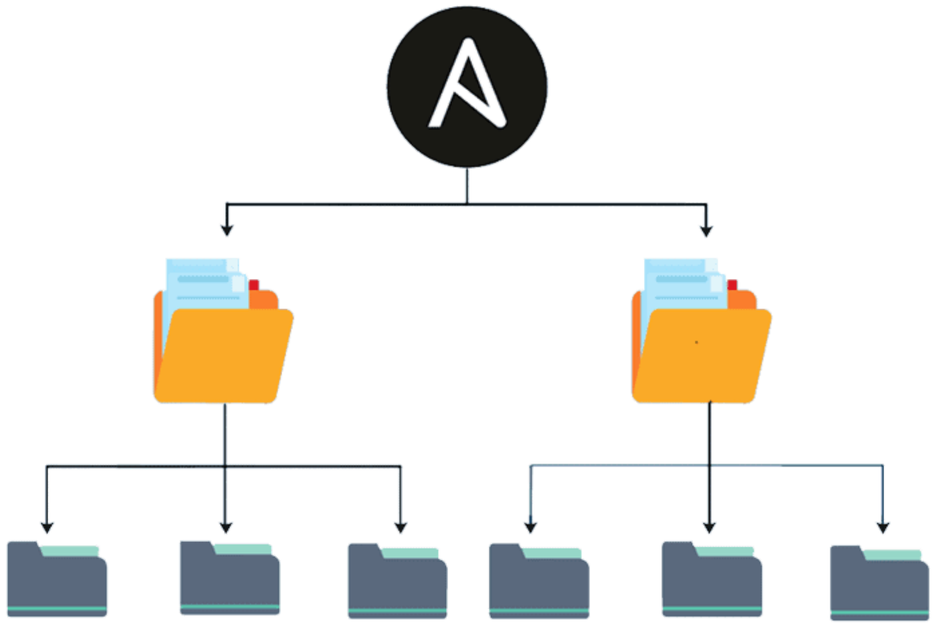
SESIÓN 5

- Estructura profesional de un rol: tasks, handlers, templates, vars y defaults.
- Refactorización del proyecto para hacerlo ordenado, reutilizable y escalable.
- Uso básico de Ansible Galaxy para aprovechar roles de la comunidad.
- Automatización de monitoreo con Prometheus y Node Exporter.

Roles

¿Qué son los Roles en Ansible?

Definición
un rol es una forma de empaquetar y organizar tareas, variables, manejadores (handlers), archivos y plantillas en una estructura de directorios estandarizada para facilitar su reutilización y modularidad.



Ventajas:

- **Reutilización:** Escribes la configuración una vez y la usas en múltiples playbooks o proyectos.
- **Organización modular:** Separa tareas, variables, plantillas y manejadores en carpetas estándar, evitando archivos gigantes.

La estructura de un rol

- **tasks/:** Aquí van los módulos (apt, copy, service). Es lo que antes tenías en la sección tasks: del playbook.
- **handlers/:** Los disparadores (restart nginx).
- **templates/:** Tus archivos .j2.
- **files/:** Archivos que se copian tal cual (certificados, imágenes).
- **defaults/:** Variables con la prioridad más baja. Están hechas para ser sobrescritas fácilmente por el usuario del rol.
- **vars/:** Variables con prioridad alta. Úsalas para constantes que rara vez cambian.
- **meta/:** Metadatos del rol: información del autor, licencia y dependencias de otros roles.

```
<role_name>/
├── defaults/
│   └── main.yml
├── files/
│   ├── file.txt
│   └── script.sh
├── handlers/
│   └── main.yml
├── library/
├── meta/
│   └── main.yml
├── module_utils/
├── plugins/
├── tasks/
│   └── main.yml
├── templates/
│   └── config.j2
├── tests/
│   ├── inventory
│   └── test.yml
├── vars/
│   └── main.yml
```

creando tu primer rol

Refactorización (1/3)

```
# old_playbook_npm_mariadb.yaml (espagueti) ❌
---
- hosts: servers
  become: true
  tasks:
    - name: Crear red personalizada de Docker
      community.docker.docker_network:
        name: npm_network
        state: present
```

Vamos a re-factorizar. Tomaremos el "código espagueti" de un servidor NPM y lo convertiremos en un rol elegante.

Refactorización (2/3)

```
    - name: Desplegar base de datos MariaDB
      community.docker.docker_container:
        name: npm_db
        image: mariadb:11.4
        state: started
        restart_policy: unless-stopped
        networks:
          - name: npm_network
        volumes:
          - /opt/npm/db:/var/lib/mysql
        env:
          MYSQL_ROOT_PASSWORD: npm_root_password
          MYSQL_DATABASE: npm
          MYSQL_USER: npm
          MYSQL_PASSWORD: npm_db_password
```

Dependencias entre roles

Definición
permiten que un rol ejecute automáticamente otros roles antes de sus propias tareas. Se declaran en el archivo meta/main.yml *dentro de la carpeta del rol.*

Ventajas:

- **Automatización transparente de prerrequisitos:** El rol principal declara qué necesita para funcionar y Ansible lo resuelve automáticamente sin que tengas que listarlo manualmente en cada playbook.
- **Playbooks limpios y concisos:** Tu playbook principal solo necesita llamar al rol de más alto nivel, manteniendo la orquestación simple

Declaración en meta/main.yml

```yaml
# roles/npm_stack/meta/main.yml
---
dependencies:
  - role: common_docker
  - role: geerlingguy.pip
    vars:
      pip_install_packages:
        - name: docker
```

Ansible Galaxy

Ansible galaxy

Definición
Es el repositorio público oficial y la herramienta de línea de comandos del ecosistema de Ansible para descubrir, descargar, compartir y gestionar contenido automatizado creado por la comunidad y fabricantes (como Red Hat, Cisco, AWS, etc.).



***Ventajas*:**

- **Ahorro de tiempo:** No reinventes la rueda. Usa roles probados en producción.
- **Calidad:** Los roles populares tienen miles de descargas y están bien mantenidos.
- **Estandarización:** Aprende buenas prácticas viendo código de expertos.
- **Comunidad:** Contribuye con mejoras y reporta bugs.

Buscando roles en Galaxy

Buscar roles relacionados con "docker"

```
ansible-galaxy search docker
```

Buscar con un término más específico

```
ansible-galaxy search mysql \
   --author geerlingguy
```

Ver detalles de un rol específico

```
ansible-galaxy info geerlingguy.docker
```

La forma más cómoda es buscar en ***galaxy.ansible.com***

Criterio	¿Qué buscar?
Popularidad	Más de 1000 descargas, estrellas altas
Mantenimiento	Última actualización reciente (< 6 meses)
Compatibilidad	Soporta tu distribución y versión de Ansible
Documentación	README completo con ejemplos
Licencia	Open source (MIT, Apache, BSD)

Instalando roles desde Galaxy

Instalar un rol (se guarda en ~/.ansible/roles/)

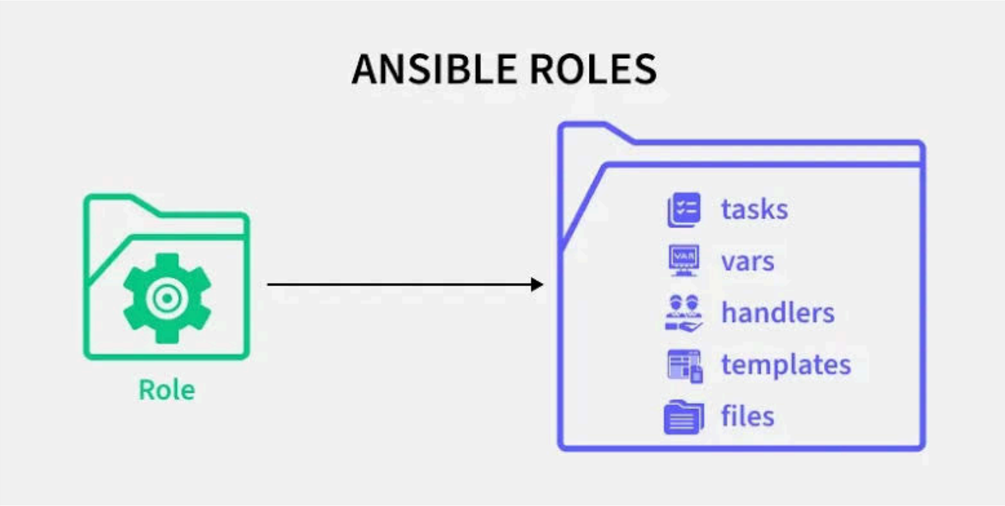
```
ansible-galaxy install geerlingguy.docker
```

Instalar en una ruta específica

```
ansible-galaxy install geerlingguy.nginx -p ./roles/
```

Instalar una versión específica

```
ansible-galaxy install geerlingguy.mysql,3.4.0
```



Usando requirements.yml

requirements.yml

```yaml
# Roles desde Galaxy
roles:
  - name: geerlingguy.docker
    version: 6.1.0

  - name: geerlingguy.nginx
    version: 3.1.4

# Collections
collections:
  - name: community.general
    version: 8.1.0
```

Instalar todos los roles y collections del archivo

```
ansible-galaxy install -r requirements.yml
```

Forzar reinstalación (útil para actualizar)

```
ansible-galaxy install -r requirements.yml --force
```

Instalar en ruta específica

```
ansible-galaxy install -r requirements.yml -p ./
roles/
```

Instalando desde Git requirements.yml

```yaml
roles:
  # Desde GitHub
  - src: https://github.com/usuario/mi-rol.git
    name: mi_rol_custom
    version: main  # Branch, tag o commit

  # Desde GitLab
  - src: git@gitlab.com:empresa/rol-interno.git
    name: rol_interno
    scm: git

  # Desde Galaxy con nombre personalizado
  - src: geerlingguy.apache
    name: apache_role
    version: 3.2.0
```

Usando roles instalados en playbooks

```yaml
- name: Configurar servidor web con roles de Galaxy
  hosts: servers
  become: true

  vars:
    # Variables para geerlingguy.docker
    docker_users:
      - deployer
      - jenkins

    # Variables para geerlingguy.nginx
    nginx_vhosts:
      - listen: "80"
        server_name: "ejemplo.com www.ejemplo.com"
        root: "/var/www/ejemplo"
```

```yaml
  roles:
    - geerlingguy.docker
    - geerlingguy.nginx
    - geerlingguy.certbot

  post_tasks:
    - name: Verificar Docker
      service:
        name: docker
        state: started
```

Ver variables disponibles de un rol:
galaxy.ansible.com

Ansible Collections

Ansible collections

Definición
es un formato de paquete para empaquetar y distribuir contenido de automatización.

paquete que puede incluir:

- **Roles:** Como los tradicionales
- **Módulos:** Nuevas funcionalidades (aws_ec2, docker_container)
- **Plugins:** Filtros, callbacks, inventarios
- **Documentación:** Guías y ejemplos

Collection	Descripción
`community.general`	Módulos generales de la comunidad
`community.kubernetes`	Kubernetes/K8s
`amazon.aws`	Servicios de AWS
`azure.azcollection`	Microsoft Azure
`google.cloud`	Google Cloud Platform
`community.docker`	Gestión de Docker

Instalando roles desde Galaxy

Instalar una collection

```
ansible-galaxy collection install community.general
```

Instalar versión específica

```
ansible-galaxy collection install amazon.aws:7.1.0
```

Listar collections instaladas

```
ansible-galaxy collection list
```



Ver información de una collection

```
ansible-galaxy collection info community.docker
```

Instalar en una ruta específica

```
ansible-galaxy collection community.docker -p ./collections/ansible_collections/
```

Usando collections en playbooks

Opción 1: Declarar a nivel de play

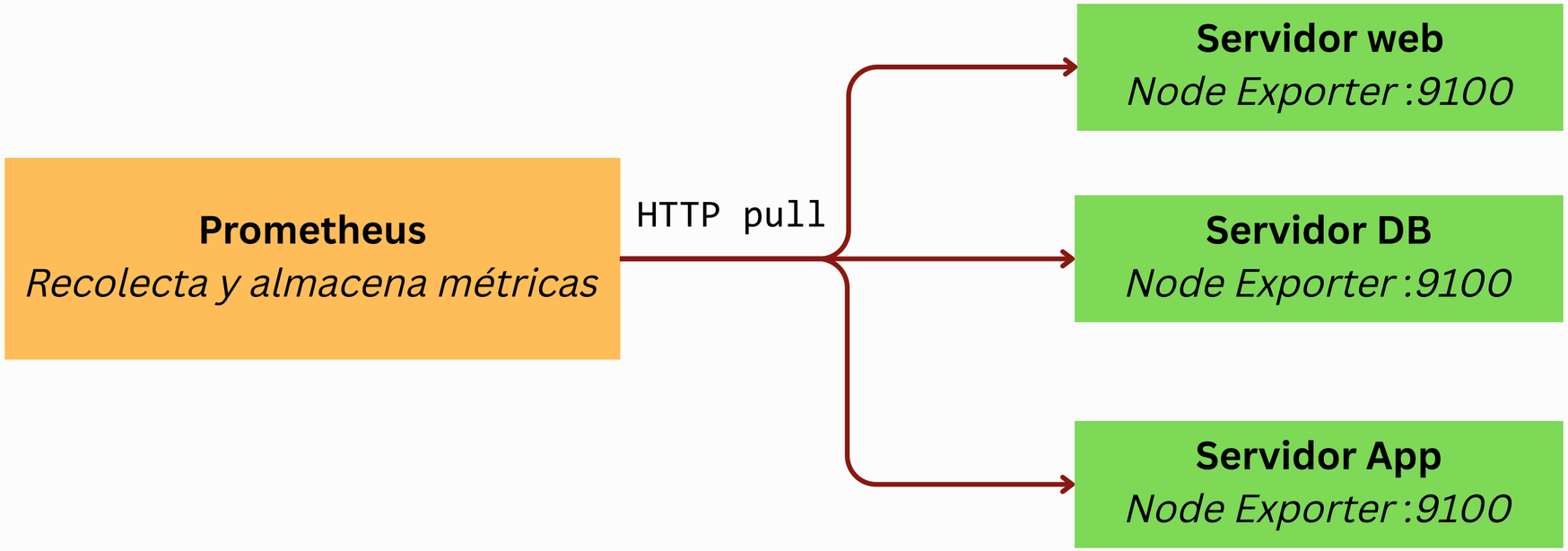
```yaml
- name: Gestionar AWS EC2
  hosts: localhost
  collections:
    - amazon.aws

  tasks:
    - name: Crear instancia EC2
      ec2_instance:  # Módulo de amazon.aws
        name: target1
        instance_type: t3.micro
        image_id: ami-0c55b159cbfafe1f0
        region: us-east-1
        key_name: mi-llave
        state: running
```

Opción 2: Usar FQCN

```yaml
- name: Gestionar Docker
  hosts: servidores
  tasks:
    - name: Crear contenedor Nginx
      community.docker.docker_container:
        name: nginx
        image: nginx:latest
        ports:
          - "80:80"
        state: started
```

Arquitectura deMonitoreo Automatizado



Modelo pull
Prometheus consulta periódicamente */metrics*; Node Exporter no necesita enviar datos al servidor central.

Rol *node_exporter*: Instalación y Servicio (1/2)

tasks/main.yml

```
- name: Crear usuario del servicio
  ansible.builtin.user:
    name: node_exporter
    system: true
    shell: /usr/sbin/nologin
- name: Instalar binario
  ansible.builtin.copy:
    src: node_exporter
    dest: /usr/local/bin/node_exporter
    owner: root
    group: root
    mode: '0755'
- name: Publicar unidad systemd
  ansible.builtin.template:
    src: node_exporter.service.j2
    dest: /etc/systemd/system/node_exporter.service
  notify: Reiniciar node exporter
```

Rol *node_exporter*: Instalación y Servicio (2/2)

templates/node_exporter.service.j2

```
[Unit]
Description=Prometheus Node Exporter
After=network-online.target

[Service]
User=node_exporter
ExecStart=/usr/local/bin/node_exporter \
--web.listen-address=:{{ node_exporter_port }}
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

Estado deseado

```
- ansible.builtin.systemd_service:
    name: node_exporter
   enabled: true
    state: started
    daemon_reload: true
```

Configurar Prometheus con los Hosts Administrados

Fragmento de prometheus.yml.j2

```
scrape_configs:
  - job_name: "linux_nodes"
    static_configs:
      - targets:
{% for host in groups['linux'] %}
          - "{{ hostvars[host].ansible_host }}:9100"
{% endfor %}
```

Ventaja del template

- El inventario es la fuente de verdad.
- Los nuevos hosts se agregan automáticamente.
- El cambio notifica un handler para recargar Prometheus.

Validación e Idempotencia del Monitoreo

Comprobaciones

```
ansible linux -b -m command \
-a "systemctl is-active node_exporter"

ansible linux -m uri -a \
"url=http://localhost:9100/metrics"

ansible-playbook -i inventory.ini \
site.yml --check --diff
```

Criterios de éxito

- Servicio ***active*** y ***enabled***.
- Puerto 9100 accesible solo desde la red autorizada.
- Target en estado ***UP*** en Prometheus.
- Segunda ejecución con ***changed=0***.

Errores Frecuentes y Buenas Prácticas

Problema	Causa habitual	Práctica recomendada
Rol monolítico	Demasiadas responsabilidades	Separar por servicio o capacidad
Variables rígidas	Valores públicos dentro de *vars/*	Exponerlos en *defaults/*
Reinicios en cada corrida	Uso de tareas directas	Aplicar *notify* y *handlers*
Galaxy impredecible	Versiones no fijadas	Mantener *requirements.yml*
Target Prometheus **DOWN**	Servicio, firewall o IP incorrectos	Validar systemd, puerto y endpoint

Gracias!!!

